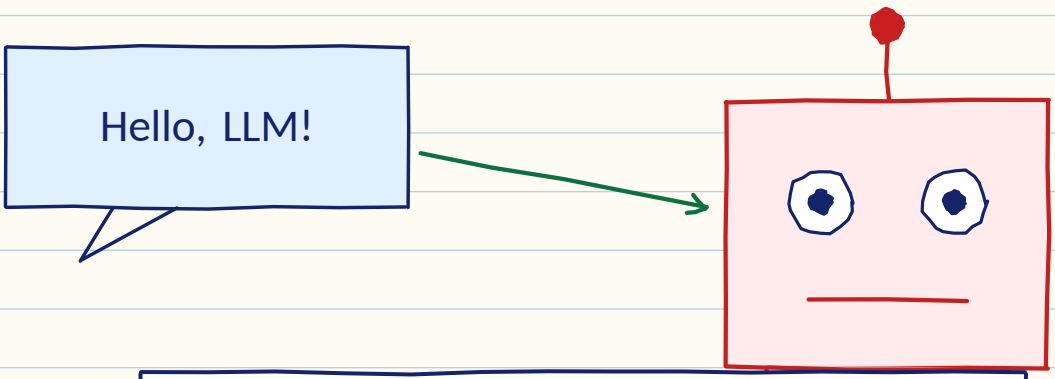
AI Agent Engineering

Day 01

Building a Basic LLM Chat Assistant

LangChain + Ollama + Qwen3:1.7B

From intuition and math to complete code logic



Handwritten Study Notes

Author: Chandan Chaudhari

GitHub: chandanc5525

github.com/chandanc5525/AI_Agent_Engineering_Sessions

Contents

1. The Big Picture 3
2. LLM Intuition: Next-Token Prediction 4
3. Tokens and Tokenization 5
4. Embeddings: Meaning as Geometry 6
5. Softmax, Logits and Temperature 8
6. Autoregressive Generation and Sampling 10
7. Attention: The Core Math 12
8. Transformer Block, Loss and Qwen3 14
9. Running Models Locally with Ollama 16
10. Project Setup: uv, .env, dotenv 18
11. LangChain Mental Model 20
12. Code Walkthrough, Line by Line 21
13. Program Flow and Execution Trace 23
14. Bug Alert: the /v1 Base URL 24
15. Stateless Calls and the Response Object 26
16. Streaming, Parameters, Context, Thinking 28
17. Improved CLI Assistant 30
18. Streamlit Version of Day 01 32
19. Troubleshooting Guide 34
20. Interview Q and A 35
21. Formula Sheet and Glossary 36
22. Exercises, Checklist, Day 02 Preview 38

How to read these notes

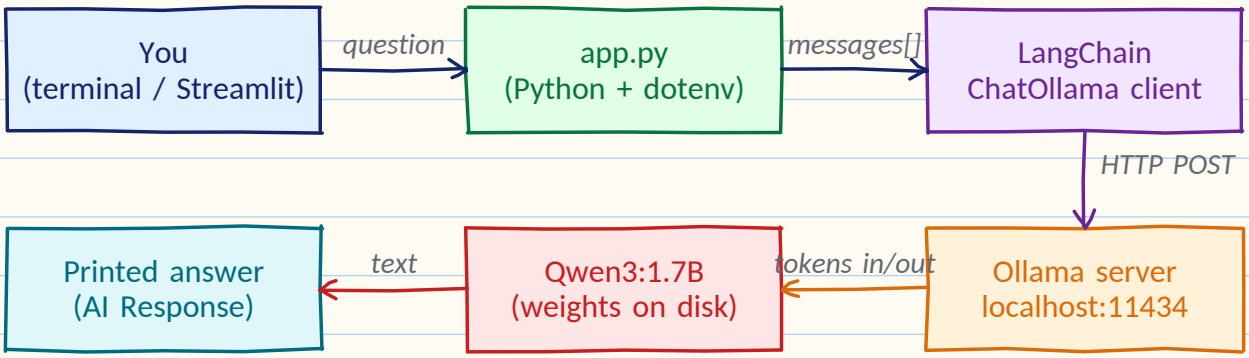
Sections 1-8 build the intuition and the math behind an LLM (with PyTorch and TensorFlow code). Sections 9-18 go through the real project: Ollama, uv, .env, LangChain and the full code. Sections 19-22 are for revision: troubleshooting, interview questions, formulas and exercises.

1. The Big Picture

Goal of Day 01: build the "brain" of an AI agent. A small Python program reads a question, sends it to a local large language model (LLM) and prints the answer. Every later day (memory, tools, RAG, multi-agent graphs) is built on top of this single call.

Chatbot vs AI Agent

	Chat assistant (Day 01)	AI agent (later)
Input	One user question at a time	A goal, plus observations from tools
Memory	None (every call is independent)	Short-term chat memory + long-term store
Actions	Only produces text	Calls tools: search, code, APIs, DB
Control	You decide the next step	The model decides the next step (loop)



Day 01 pipeline: everything runs on my own machine, no cloud key needed

After Day 01 I should be able to ...

- explain, with simple math, how an LLM turns a prompt into the next token
- run Qwen3:1.7B locally through Ollama and know which port and endpoints it exposes
- manage configuration with a .env file and python-dotenv
- call the model from Python with LangChain ChatOllama and HumanMessage
- explain why the loop has no memory and why the /v1 URL breaks

ChatOllama

Remember

Day 01 is only "LLM in, text out". No memory, no tools, no planning yet. That is exactly why it is the right foundation: I can see the raw model behaviour.

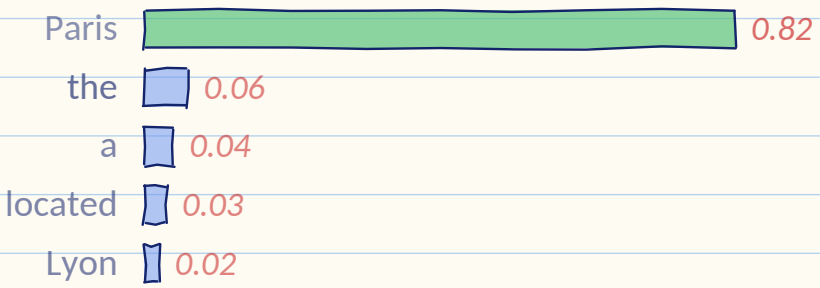
2. LLM Intuition: Next-Token Prediction

An LLM is a very large function that reads a sequence of tokens and returns a probability for every possible next token. It is autocomplete on steroids: the phone keyboard suggests 3 words, an LLM scores about 150,000 candidates and does it again and again.

P(next token | all previous tokens)

the only thing the model ever outputs

Prompt: "The capital of France is ___"



(illustrative numbers)

Training vs inference

- Training: read trillions of tokens, predict the next token, measure the error, adjust the billions of weights by gradient descent. Done once by the model makers (Alibaba for Qwen).
- Inference: weights are frozen. I only run the forward pass: prompt in, probabilities out, pick a token, repeat.
- Day 01 is 100% inference. I never train anything.

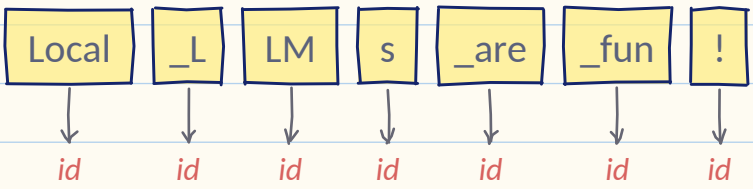
Why hallucination happens

The training goal is "predict plausible text", not "be true". When the model does not know a fact it still produces the most plausible-sounding continuation. Small models such as 1.7B hallucinate more than big ones, so later days add RAG (retrieval) and tools to give the model real facts.

> **Analogy I use in interviews**
> A student who read the whole internet but has no search engine. He answers fluently from memory; when memory is fuzzy he still answers confidently. Agents give him a library and a calculator.

3. Tokens and Tokenization

Models do not read characters or words. Text is cut into tokens (pieces of words) by a tokenizer, usually Byte-Pair Encoding (BPE). Each token maps to an integer id; the model only sees integers.



"Local LLMs are fun!" -> 7 tokens (split is illustrative; "_" marks a leading space)

BPE intuition

1. Start with single bytes/characters as the vocabulary.
2. Count the most frequent adjacent pair in the training text.
3. Merge that pair into one new token. Repeat until the vocabulary has the wanted size (about 150k for Qwen).
4. Frequent words become one token; rare words split into several pieces, so nothing is out-of-vocabulary.

Why tokens matter for me

- Speed is measured in tokens per second, not words per second.
- Context window (how much the model can read at once) is counted in tokens.
- Rule of thumb for English: 1 token is about 4 characters or 0.75 words. 1000 words is about 1300 tokens.
- Code, numbers and non-English text usually need more tokens per word.

Check

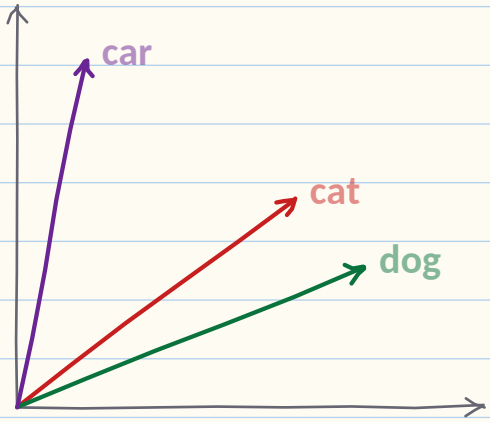
Different models use different tokenizers. The same sentence can be 7 tokens in one model and 9 in another. Never compare token counts across model families.

4. Embeddings: Meaning as Geometry

A token id is just a number, so the first layer is a lookup table (the embedding matrix) that turns each id into a vector of d numbers. Training pushes tokens used in similar contexts to similar directions. Meaning becomes geometry.

$$E \in R^{V \times d} \text{ token id } t \rightarrow \text{vector } e_t = E[t]$$

V = vocabulary size, d = hidden size



cos(cat, dog) = 0.97 angle about 15 deg

cos(cat, car) = 0.75 angle about 42 deg

Small angle = similar meaning.

(2-D toy vectors; real ones have thousands of dimensions)

Cosine similarity

$$\cos(a, b) = (a \cdot b) / (|a| \ |b|)$$

dot product divided by the lengths, range -1 to +1

Worked example: cat = (4, 3), dog = (5, 2). Dot = 4×5 + 3×2 = 26. Lengths: 5 and 5.385. So cos = 26 / 26.93 = 0.966. With car = (1, 5): dot = 19, |car| = 5.099, cos = 0.745.

PyTorch: cosine similarity

```
import torch
import torch.nn.functional as F

cat = torch.tensor([4.0, 3.0])
dog = torch.tensor([5.0, 2.0])
car = torch.tensor([1.0, 5.0])

print(F.cosine_similarity(cat, dog, dim=0))   # tensor(0.9656)
print(F.cosine_similarity(cat, car, dim=0))   # tensor(0.7452)
```

TensorFlow: cosine similarity

```
import tensorflow as tf

cat = tf.constant([4.0, 3.0])
dog = tf.constant([5.0, 2.0])
car = tf.constant([1.0, 5.0])

def cos(a, b):
    a, b = tf.nn.l2_normalize(a), tf.nn.l2_normalize(b)
    return tf.reduce_sum(a * b)

print(cos(cat, dog).numpy())   # 0.9656
print(cos(cat, car).numpy())   # 0.7452
```

5. Softmax, Logits and Temperature

The last layer outputs one raw score per vocabulary token, called a logit. Logits can be any real number, so we convert them to probabilities with softmax.

$$\text{softmax}(z)_i = e^{z_i} / \Sigma_j\, e^{z_j}$$

e^z for the i-th logit, divided by the sum over all logits

Worked example

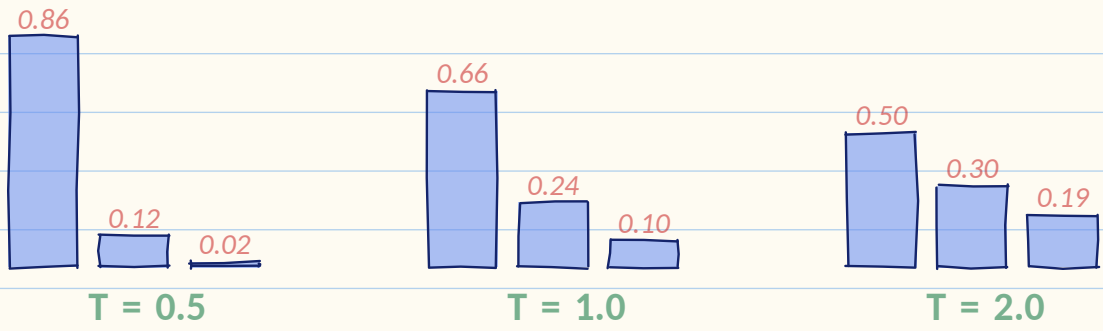
Logits z = [2.0, 1.0, 0.1]. Exponentials: e^2.0 = 7.389, e^1.0 = 2.718, e^0.1 = 1.105. Sum = 11.212. Probabilities = [0.659, 0.242, 0.099]. They are positive and add up to 1.

Temperature T

$$P_i(T) = \text{softmax}(\, z \,/\, T \,)_i$$

divide the logits by T before softmax

- T < 1 sharpens the distribution: the top token dominates, output is focused and repeatable.
- T = 1 is the raw model distribution.
- T > 1 flattens it: more randomness and creativity, but more mistakes.
- T -> 0 becomes greedy decoding (always argmax).



Computed from the same logits [2.0, 1.0, 0.1]. Lower T makes the leader win by a larger margin.

PyTorch: temperature softmax

```
import torch
import torch.nn.functional as F

logits = torch.tensor([2.0, 1.0, 0.1])
for T in (0.5, 1.0, 2.0):
    probs = F.softmax(logits / T, dim=-1)
    print(T, probs.numpy().round(3))
# 0.5 [0.864 0.117 0.019]
# 1.0 [0.659 0.242 0.099]
# 2.0 [0.502 0.304 0.194]
```

TensorFlow: temperature softmax

```
import tensorflow as tf

logits = tf.constant([2.0, 1.0, 0.1])
for T in (0.5, 1.0, 2.0):
    probs = tf.nn.softmax(logits / T)
    print(T, probs.numpy().round(3))
```

Numerical stability trick

Frameworks compute softmax(z - max(z)). Subtracting the maximum does not change the result (the factor cancels) but prevents e^z from overflowing for large logits.

6. Autoregressive Generation and Sampling

To generate a sentence, the model repeats one step. This is called autoregressive generation: its own output becomes part of the next input.

$$P(x_1, \ldots, x_n) = \Pi_{t=1..n} P(x_t \mid x_{1..t-1})$$

chain rule of probability

1. Tokenize the prompt into ids.
2. Forward pass gives logits for the next position.
3. Apply temperature, softmax, and a sampling rule to choose one token.
4. Append the chosen token to the sequence.
5. Stop at the end-of-sequence token or at the max token limit; otherwise go to step 2.

Decoding strategies

Strategy	Rule	Effect
Greedy	Always take argmax	Deterministic, can repeat itself
Temperature	Rescale logits by 1/T	Controls randomness
Top-k	Keep the k most likely tokens, renormalize, sample	Cuts the long tail
Top-p (nucleus)	Keep the smallest set whose probability sums to p	Adaptive cut-off

In ChatOllama these map to temperature, top_k, top_p, num_predict (max new tokens) and seed.

PyTorch: the autoregressive loop (toy)

```
import torch

vocab = ["I", "love", "AI", "agents", "<eos>"]
emb = torch.nn.Embedding(len(vocab), 8)
head = torch.nn.Linear(8, len(vocab))

ids = [0]                                   # start with "I"
for _ in range(6):
    logits = head(emb(torch.tensor(ids[-1])))
    probs = torch.softmax(logits, dim=-1)
    nxt = torch.multinomial(probs, 1).item()   # sampling
    ids.append(nxt)
    if vocab[nxt] == "<eos>":
        break
print(" ".join(vocab[i] for i in ids))
# untrained weights -> random words; this is only the LOOP shape
```

TensorFlow: the autoregressive loop (toy)

```
import tensorflow as tf

vocab = ["I", "love", "AI", "agents", "<eos>"]
emb = tf.keras.layers.Embedding(len(vocab), 8)
head = tf.keras.layers.Dense(len(vocab))

ids = [0]
for _ in range(6):
    x = emb(tf.constant([ids[-1]]))                 # shape (1, 8)
    logits = head(x)                                # shape (1, 5)
    nxt = int(tf.random.categorical(logits, 1)[0, 0])
    ids.append(nxt)
    if vocab[nxt] == "<eos>":
        break
print(" ".join(vocab[i] for i in ids))
```

Key insight

Each new token needs one forward pass. A 200-token answer = 200 forward passes. That is why long answers feel slow and why caching the past (KV cache) matters.

7. Attention: The Core Math

Attention lets every token look at earlier tokens and decide which ones matter. Example: in "The cat sat because it was tired", the token "it" must attach to "cat". Attention computes that link.

Query, Key, Value intuition

- Query (Q): what am I looking for?
- Key (K): what do I contain, as a label others can match?
- Value (V): the actual information I hand over if I am selected.
- Each token vector x is turned into Q, K, V by three learned matrices: Q = xW_Q, K = xW_K, V = xW_V.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q K^T / \sqrt{(d_k)} \right) V$$

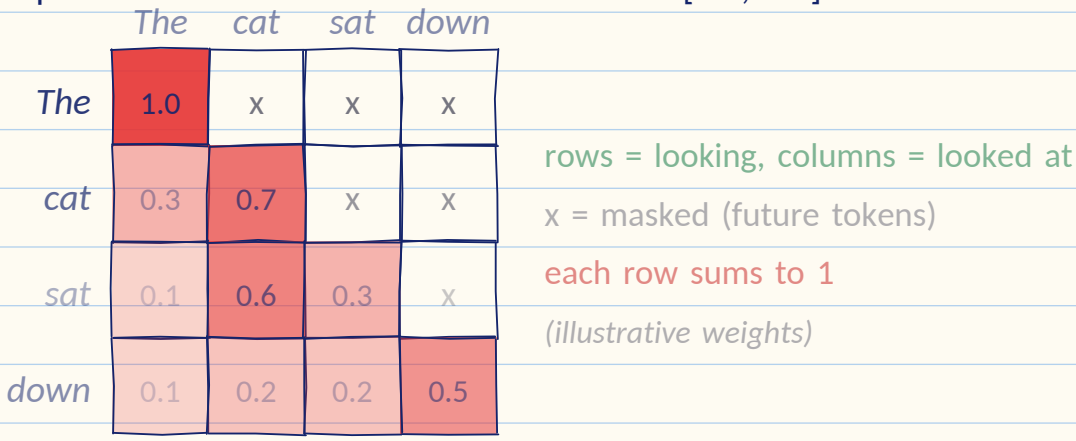
the most important formula in modern AI

- Q K^T: dot product of every query with every key gives a similarity score matrix.
- Divide by sqrt(d_k): keeps scores from growing with dimension, which would saturate softmax.
- softmax: turns each row into weights that sum to 1.
- Multiply by V: a weighted average of values is the new token vector.
- Causal mask: a token may only see itself and earlier tokens (upper triangle set to minus infinity).

Tiny worked example (d_k = 2)

q = [1, 0]. Keys: k1 = [1, 0], k2 = [0, 1], k3 = [1, 1]. Values: v1 = [1, 2], v2 = [3, 4], v3 = [5, 6].

1. Scores q·k = [1, 0, 1]; divide by sqrt(2) = 1.414 -> [0.707, 0, 0.707].
2. Softmax -> [0.401, 0.198, 0.401].
3. Output = 0.401×v1 + 0.198×v2 + 0.401×v3 = [3.0, 4.0].



PyTorch: scaled dot-product attention

```
import torch

Q = torch.tensor([[1.0, 0.0]])
K = torch.tensor([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
V = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])

d_k = K.shape[-1]
scores = Q @ K.T / d_k ** 0.5          # (1, 3)
weights = torch.softmax(scores, dim=-1)
out = weights @ V                      # (1, 2)
print(weights)   # [[0.4011, 0.1978, 0.4011]]
print(out)       # [[3., 4.]]
```

TensorFlow: scaled dot-product attention

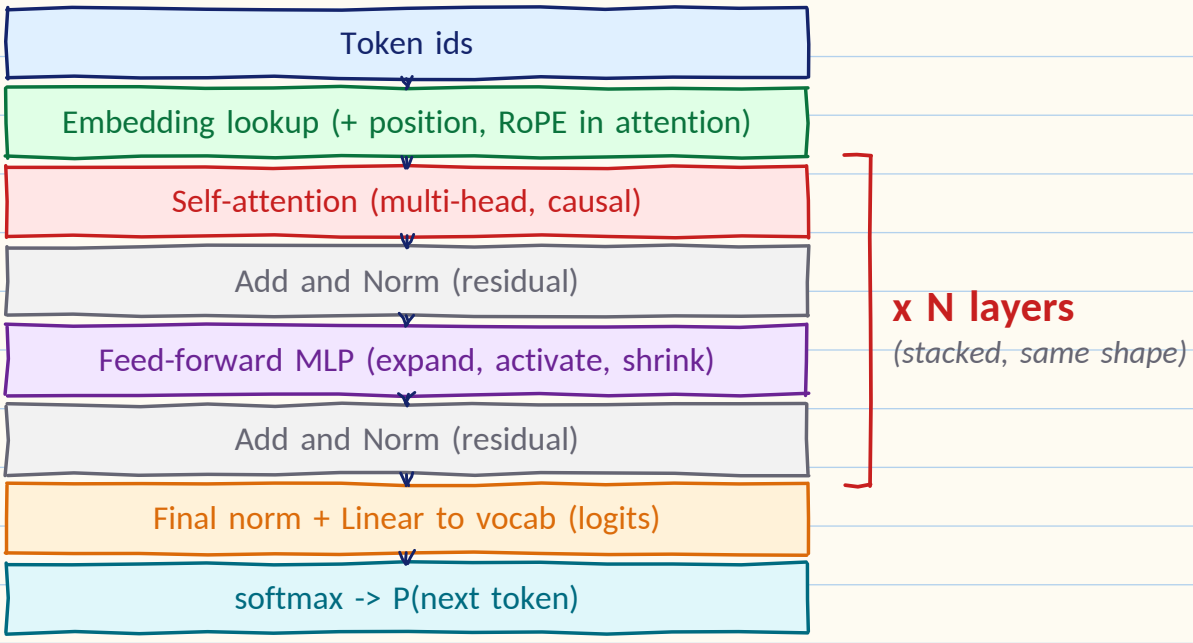
```
import tensorflow as tf

Q = tf.constant([[1.0, 0.0]])
K = tf.constant([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
V = tf.constant([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])

d_k = tf.cast(tf.shape(K)[-1], tf.float32)
scores = tf.matmul(Q, K, transpose_b=True) / tf.sqrt(d_k)
weights = tf.nn.softmax(scores, axis=-1)
out = tf.matmul(weights, V)
print(weights.numpy())   # [[0.4011 0.1978 0.4011]]
print(out.numpy())       # [[3. 4.]]
```

Multi-head attention: run h attention heads in parallel with smaller dimensions, each learning a different relation (grammar, coreference, position...), then concatenate.

8. Transformer Block, Loss and Qwen3



Training objective: cross-entropy

For the true next token y the loss is minus the log of the probability the model gave it. Confident and right -> loss near 0. Confident and wrong -> huge loss.

L = - log P(y | context) (averaged over all positions)

example: P = 0.659 -> L = 0.417; P = 0.099 -> L = 2.317

PyTorch: cross-entropy loss

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 1.0, 0.1]])
target = torch.tensor([0])                  # the true token id is 0
loss = F.cross_entropy(logits, target)      # softmax + -log inside
print(loss.item())                          # 0.4170
```

TensorFlow: cross-entropy loss

```
import tensorflow as tf

logits = tf.constant([[2.0, 1.0, 0.1]])
labels = tf.constant([0])
loss = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=labels, logits=logits)
print(loss.numpy())                         # [0.417]
```

What "1.7B" means

- About 1.7 billion learned parameters (numbers) inside the embedding, attention and MLP matrices.
- Memory = parameters × bytes per parameter. FP16: 1.7e9 × 2 bytes = 3.4 GB. 4-bit quantized: 1.7e9 × 0.5 byte = 0.85 GB raw (about 1-1.5 GB in practice with overhead).
- That is why a 1.7B model runs on a laptop CPU, while a 70B model needs a big GPU.

KV cache (why long chats cost memory)

To avoid recomputing K and V of old tokens at every step, the model stores them: the KV cache.

KV bytes per token = 2 × layers × kv_heads × head_dim × bytes

illustrative: 2×28×8×128×2 = 114,688 B (about 112 KB) per token; 4096 tokens -> about 470 MB

> **Note**
> The layer and head numbers above are illustrative values for a model of this size. Check the official model card for the exact architecture.

About Qwen3

- Open-weight model family from Alibaba (Qwen team); this repo uses the small qwen3:1.7b tag.
- Supports a thinking mode: it can emit its reasoning inside think tags before the final answer (details in Section 16).
- Small models are ideal for learning: fast, free, private, and good enough to see agent patterns.

9. Running Models Locally with Ollama

Ollama is a local model runner. It downloads model weights, loads them into RAM/VRAM, and serves them through an HTTP server on port 11434. My Python code never touches weights; it only sends HTTP requests.

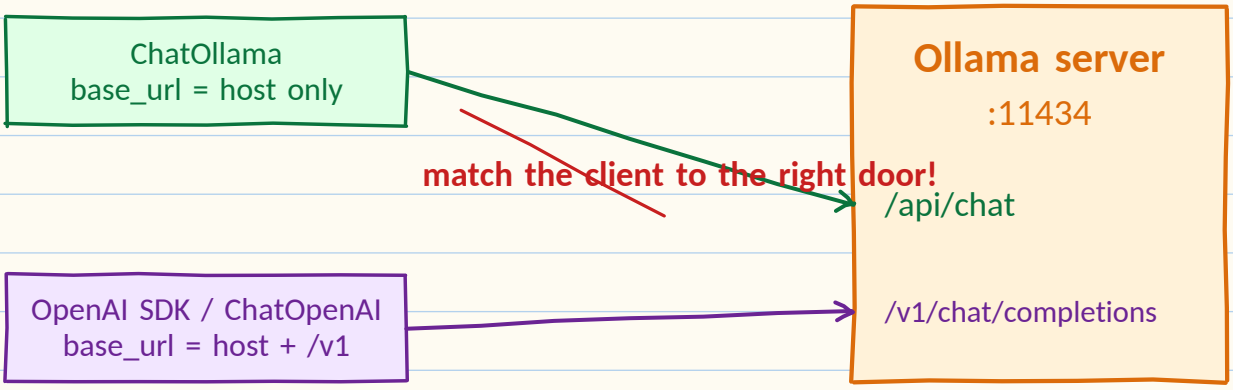
Essential commands

Terminal cheat sheet

```
ollama --version              # is it installed?
ollama pull qwen3:1.7b        # download weights (about 1.4 GB)
ollama list                   # models available locally
ollama run qwen3:1.7b         # chat directly in the terminal
ollama ps                     # models currently loaded in memory
ollama serve                  # start the server manually
ollama rm qwen3:1.7b          # delete a model
```

Two APIs, one server

API style	Base URL	Used by
Native Ollama API: /api/chat, /api/generate	http://localhost:11434	ChatOllama, ollama python library
OpenAI-compatible: /v1/chat/completions	http://localhost:11434/v1	OpenAI SDK, ChatOpenAI, many tools



Quantization in one paragraph

Weights are normally 16-bit floats. Quantization stores them in about 4 bits with a scale factor, shrinking memory roughly 3-4× with a small quality loss. Ollama tags are typically quantized, which is how a billion-parameter model fits in a laptop. Speed on CPU is limited by memory bandwidth because each generated token reads all the weights once.

Quick sanity test

Before writing any Python, run: ollama run qwen3:1.7b and type "hello". If this works, the model and server are fine and any later error is in my code or configuration.

10. Project Setup: uv, .env, dotenv

uv: fast Python project manager

The repo uses uv (from Astral) for virtual environments and dependency locking. It replaces pip + venv + pip-tools with one fast tool. pyproject.toml lists dependencies; uv.lock pins exact versions so that every machine installs the same thing; .python-version pins the interpreter.

Terminal: uv workflow

```
uv init                                  # create pyproject.toml
uv add langchain-ollama python-dotenv
uv add langchain-core langgraph          # later days
uv sync                                  # install from uv.lock
uv run python main.py                    # run inside the project env
uv run streamlit run app.py              # run a Streamlit app
```

Folder layout

Project tree (as in the repo)

```
AI_Agent_Engineering_Sessions/
|-- .env                    # secrets and settings (never commit!)
|-- .gitignore              # contains .env, .venv
|-- .python-version
|-- pyproject.toml
|-- uv.lock
|-- main.py
|-- Day01/ ... Day04/       # one folder per session, each with app.py
`-- PythonProgramming_BootCamp/
```

.env and python-dotenv

Configuration that changes between machines (model name, URL, API keys) should not be hard-coded. It lives in environment variables. A `.env` file is a plain text list of KEY=VALUE lines; `load_dotenv()` reads it and puts the pairs into `os.environ`, and `os.getenv("KEY", default)` reads them.

.env (from the repo README)

```
BASE_URL=http://localhost:11434/v1
API_KEY=ollama
MODEL=qwen3:1.7b
```

- No spaces around "=" and normally no quotes.
- `os.getenv("MODEL", "qwen3:1.7b")` returns the default when the variable is missing, so the app still starts.
- Add `.env` to `.gitignore`. Real API keys must never reach GitHub. Commit a `.env.example` instead.
- `API_KEY=ollama` is a dummy value: Ollama needs no key, but the OpenAI SDK refuses an empty one.

12-factor idea

Keep config in the environment, not in code. Same code, different .env on laptop, test server and production.

11. LangChain Mental Model

Why LangChain at all? I could call the Ollama HTTP API by hand. LangChain gives one standard interface for every provider (Ollama, OpenAI, Anthropic, Gemini...). Swapping the model later is a one-line change, and the same objects plug into prompts, tools, retrievers and LangGraph.

Messages: the unit of conversation

Class	Role	Meaning
SystemMessage	system	Rules and persona for the model
HumanMessage	user	What the person typed
AIMessage	assistant	What the model answered
ToolMessage	tool	Result of a tool call (agents, later)

A chat model takes a list of messages and returns one AIMessage. Even a single question is wrapped as [HumanMessage(content=...)].

The Runnable interface

Every LangChain component (models, prompts, parsers, retrievers) is a Runnable with the same methods:

Method	Does	Day 01 use
invoke(x)	One input, one full output	Used in the loop
stream(x)	Yields output chunks as generated	Better UX (Section 16)
batch([x, y])	Many inputs in parallel	Later
ainvoke / astream	Async versions	Web servers

Packages I installed and why

- langchain-ollama: provides ChatOllama (talks to the Ollama native API).
- langchain-core: base types such as HumanMessage, SystemMessage, Runnable.
- python-dotenv: reads .env.

- langgraph (later): stateful, looping agent graphs.

Mental model

ChatOllama = translator. It turns my Python message objects into the JSON that Ollama understands, sends it, and turns the JSON reply back into an AIMessage.

12. Code Walkthrough, Line by Line

My program, with comments shortened. Below it I explain each part.

main.py (my Day 01 code)

```
import os
from dotenv import load_dotenv
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage

load_dotenv()                                    # 1. read .env

model_name = os.getenv("MODEL", "qwen3:1.7b")  # 2. settings
base_url = os.getenv("BASE_URL", "http://localhost:11434")

llm = ChatOllama(                                # 3. model client
    model=model_name,
    base_url=base_url
)

print('*' * 40)
print('My LangChain Ollama Assistant')
print('*' * 40)

while True:                                      # 4. chat loop
    user_input = input("\nEnter your question (or type 'exit'): ")

    if user_input.lower() == 'exit':
        print("Exiting the assistant. Goodbye!")
        break

    messages = [HumanMessage(content=user_input)]
```

```
    response = llm.invoke(messages)            # 5. call the model

    print("\nAI Response:")
    print(response.content)                    # 6. show the answer
```

Imports

- os: standard library, access to environment variables.
- load_dotenv: loads the .env file into the environment.
- ChatOllama: the chat-model client for a local Ollama server.
- HumanMessage: wraps the typed text as a "user" role message.

Step 1 to 3: configuration and client

- load_dotenv() must run before os.getenv, otherwise the variables are not there yet.
- os.getenv("MODEL", "qwen3:1.7b"): read MODEL, fall back to the default.
- ChatOllama(model=..., base_url=...): creates the client object. No network call happens yet; the first request is sent at invoke.
- Important: ChatOllama does not read the .env by itself. My own os.getenv lines do that and pass the values in.
- API_KEY is read by nobody here. ChatOllama has no key parameter for a local server.

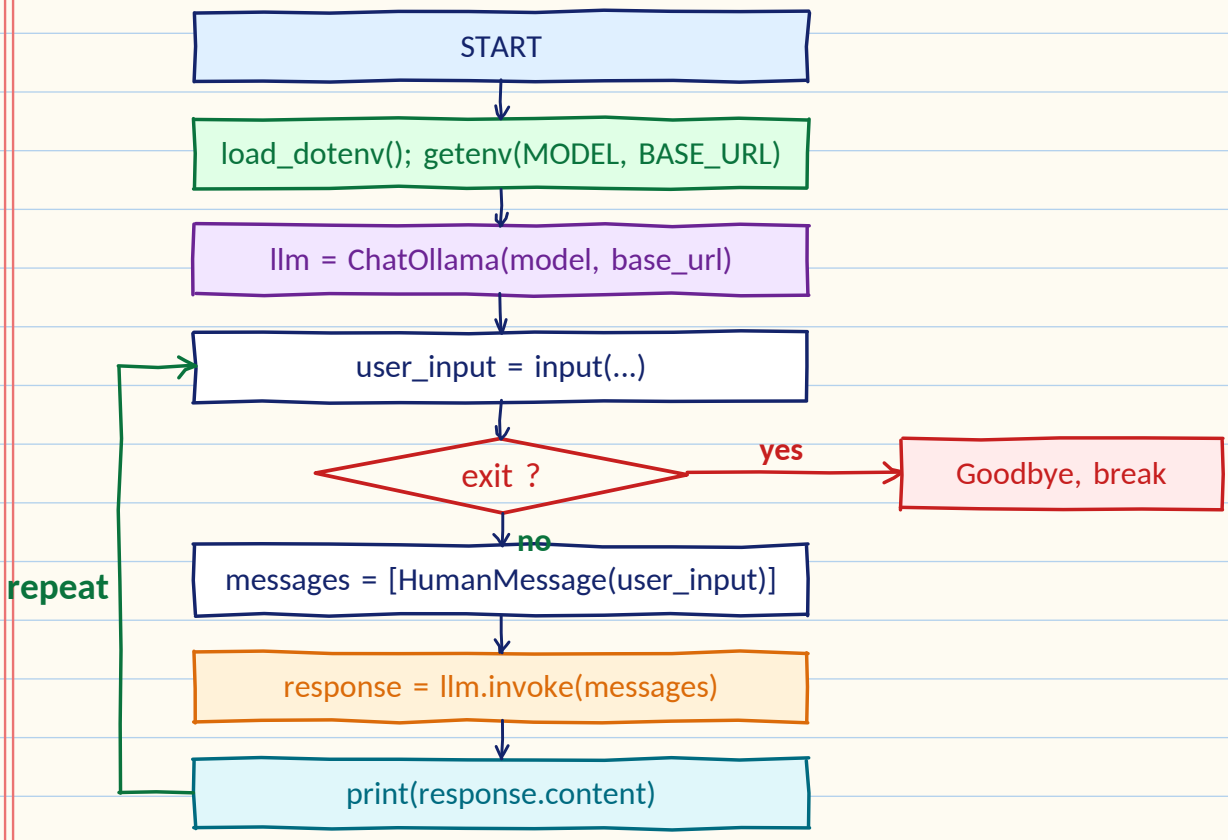
Step 4 to 6: the loop

- while True: runs until break. Typing "exit" ends it.
- input() blocks and returns a string. .lower() makes "EXIT" and "Exit" work.
- HumanMessage(content=user_input) + the list brackets = the exact input type a chat model expects.
- llm.invoke(messages) sends the HTTP request, waits for the entire answer, returns an AIMessage.
- response.content is the answer text (a string). Other fields hold metadata (Section 15).

Small observations

The "\n" at the start of the prompt only adds a blank line. Empty input is sent to the model as-is (wasteful). Connection errors crash the loop. The improved version in Section 17 fixes both.

13. Program Flow and Execution Trace



Execution trace

Sample session (output text will differ)

```
$ uv run python main.py
****************************************
My LangChain Ollama Assistant
****************************************

Enter your question (or type 'exit'): What is an AI agent?

AI Response:
An AI agent is a system that uses an LLM to decide ...

Enter your question (or type 'exit'): exit
Exiting the assistant. Goodbye!
```

- The first request is the slowest: Ollama must load the weights from disk into RAM (cold start).
- The model then stays loaded for a few minutes by default, so later questions respond faster.
- Time per answer is about: prompt processing + (tokens generated ÷ tokens per second).

14. Bug Alert: the /v1 Base URL

Likely bug in my setup

My .env has BASE_URL=http://localhost:11434/v1, which is the address for the OpenAI-compatible API. But ChatOllama talks to the native Ollama API at /api/chat. It appends /api/chat to base_url, so the request goes to /v1/api/chat, which does not exist. Typical symptom: a 404 error or "model not found" even though the model is installed.

Why did I write /v1 in the first place? The README stack uses the OpenAI SDK against Ollama, and that client needs the /v1 path. A single variable cannot serve both clients, so the safest fix is to normalise it in code.

Fix A (recommended): strip /v1 for ChatOllama

main.py

```
base_url = os.getenv("BASE_URL", "http://localhost:11434")
base_url = base_url.rstrip("/").removesuffix("/v1")   # Python 3.9+

llm = ChatOllama(model=model_name, base_url=base_url)
```

Fix B: edit .env to host only

.env (for ChatOllama only)

```
BASE_URL=http://localhost:11434
API_KEY=ollama
MODEL=qwen3:1.7b
```

Fix C: keep /v1 and use the OpenAI-compatible client

ChatOpenAI against Ollama

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI   # uv add langchain-openai
from langchain_core.messages import HumanMessage

load_dotenv()
llm = ChatOpenAI(
    model=os.getenv("MODEL", "qwen3:1.7b"),
    base_url=os.getenv("BASE_URL", "http://localhost:11434/v1"),
    api_key=os.getenv("API_KEY", "ollama"),
)
print(llm.invoke([HumanMessage(content="Hi")]).content)
```

Option	Pros	Cons
A: strip /v1 in code	Same .env works for every client	One extra line
B: host-only .env	Simplest, no code change	Breaks the OpenAI SDK scripts
C: ChatOpenAI + /v1	Same code works with real OpenAI later	Extra package; fewer Ollama-only options

Debug trick

Test the server directly: curl http://localhost:11434/api/tags lists installed models. If that works but Python fails, the problem is the URL path, not Ollama.

15. Stateless Calls and the Response Object

Each invoke is an independent request. The model keeps no state between calls; my loop builds a fresh [HumanMessage] every time, so earlier turns are simply not sent.

Turn 1

payload sent:
[Human: "My name is Chandan."]

You: My name is Chandan.

AI: Nice to meet you, Chandan!

forgot!

Turn 2

payload sent:
[Human: "What is my name?"]

You: What is my name?

AI: Sorry, I don't know your name.

This is by design: the model is a stateless function. Memory is an application feature: I must resend the earlier messages myself. That is Day 02 (conversation memory).

Anatomy of the response object

Inspecting an AIMessage

```
response = llm.invoke(messages)

print(type(response))                # AIMessage
print(response.content)              # the answer text
print(response.usage_metadata)       # input/output/total tokens
print(response.response_metadata)    # timings, done_reason, model
```

Field	Meaning
content	Answer text (string)
usage_metadata	input_tokens, output_tokens, total_tokens
response_metadata	Ollama extras: total_duration, load_duration, prompt_eval_count, eval_count, eval_duration, done_reason
id / type	Message id and the type name

Measuring speed

Ollama reports durations in nanoseconds. Generation speed is the number of generated tokens divided by the generation time.

$$\text{tokens per second} = \text{eval_count} \;/\; (\; \text{eval_duration} \;/\; 10^9 \;)$$

example: 200 tokens in 8 s of generation -> 25 tokens/s

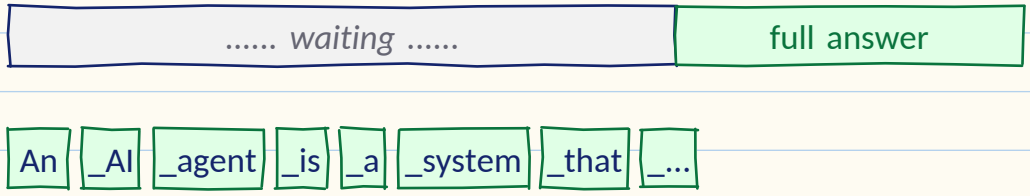
Compute tokens per second

```
meta = response.response_metadata
tps = meta["eval_count"] / (meta["eval_duration"] / 1e9)
print(f"{tps:.1f} tokens/second")
```

16. Streaming, Parameters, Context, Thinking

Streaming

invoke returns only after the whole answer exists. stream yields small chunks as soon as each token is generated, so the user sees text appear immediately. Total time is the same, but it feels much faster.



Streaming loop

```
for chunk in llm.stream([HumanMessage(content=user_input)]):
    print(chunk.content, end="", flush=True)
print()
```

Model parameters worth knowing

Parameter	Meaning	Typical use
temperature	Randomness (Section 5)	0-0.3 facts and code; 0.7+ creative
top_p / top_k	Restrict sampling to likely tokens	Leave default unless needed
num_predict	Max new tokens	Cap runaway answers
num_ctx	Context window in tokens	Raise for long chats and documents
seed	Fix the random seed	Reproducible experiments
reasoning	Control Qwen3 think output (recent langchain-ollama versions)	False for clean answers

num_ctx (total tokens the model can see at once)



- If prompt + answer exceed `num_ctx`, the oldest tokens are dropped or generation stops: the model "forgets" the start.
- Ollama's default context is small (a few thousand tokens, depends on version), so set `num_ctx` explicitly when needed.
- Bigger context uses more RAM (the KV cache from Section 8) and is slower.

Qwen3 thinking mode

Qwen3 can "think out loud" before answering, in a block marked with think tags. This improves reasoning but costs extra tokens and time. Depending on the langchain-ollama version and settings, that reasoning may show up inside `response.content` or in a separate field.

- Switch it off per message by adding `/no_think` to the prompt (Qwen3 soft switch).
- Or use the `reasoning` parameter of ChatOllama if your installed version supports it.
- Or strip the block with a regex before displaying (code below).

Remove <think> blocks

```
import re

def strip_think(text: str) -> str:
    pat = r"<think>.*?</think>"
    return re.sub(pat, "", text, flags=re.DOTALL).strip()

print(strip_think(response.content))
```

17. Improved CLI Assistant

Same idea as my loop, hardened for real use: URL normalisation, system prompt, streaming, empty-input guard, friendly errors and a clean main() entry point.

main.py (improved)

```
import os
from dotenv import load_dotenv
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage, SystemMessage

load_dotenv()

MODEL = os.getenv("MODEL", "qwen3:1.7b")
HOST = os.getenv("BASE_URL", "http://localhost:11434")
HOST = HOST.rstrip("/").removesuffix("/v1")  # host only

llm = ChatOllama(
    model=MODEL,
    base_url=HOST,
    temperature=0.3,
    num_ctx=4096,
)

SYSTEM = SystemMessage(content="You are a concise assistant.")


def ask(question: str) -> None:
    messages = [SYSTEM, HumanMessage(content=question)]
    print("\nAI Response:")
    for chunk in llm.stream(messages):
        print(chunk.content, end="", flush=True)
    print()


def main() -> None:
    print("=" * 40)
    print("My LangChain Ollama Assistant")
    print("=" * 40)
```

```
    while True:
        try:
            q = input("\nYour question (or 'exit'): ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nGoodbye!")
            break
        if q.lower() in {"exit", "quit"}:
            print("Exiting the assistant. Goodbye!")
            break
        if not q:
            continue
        try:
            ask(q)
        except Exception as err:
            print(f"\n[error] {err}")
            print("Is Ollama running? Did you run 'ollama pull'?")


if __name__ == "__main__":
    main()
```

What changed and why

Change	Reason
removesuffix("/v1")	One .env works for OpenAI SDK and ChatOllama
SystemMessage	Sets behaviour and tone before the user speaks
llm.stream(...)	Text appears immediately
.strip() + empty check	No wasted model calls on blank input
try / except	Server down does not crash the program
if __name__ == "__main__"	File can be imported without running the loop
num_ctx, temperature	Explicit, reproducible behaviour

Still missing (on purpose)

Memory of earlier turns. Adding a list that grows with each Human and AI message is the whole of Day 02.

18. Streamlit Version of Day 01

The repo README runs Day 01 with `uv run streamlit run app.py`. Streamlit turns a Python script into a web chat page. The listing below is my reference implementation of the same idea (it is not a copy of the repo file).

Day01/app.py (reference version)

```
import os
import streamlit as st
from dotenv import load_dotenv
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage

load_dotenv()
MODEL = os.getenv("MODEL", "qwen3:1.7b")
HOST = os.getenv("BASE_URL", "http://localhost:11434")
HOST = HOST.rstrip("/").removesuffix("/v1")


@st.cache_resource
def get_llm():
    return ChatOllama(model=MODEL, base_url=HOST)


st.set_page_config(page_title="Day 01 - AI Agent")
st.title("Day 01: Basic AI Agent")
st.caption(f"Model: {MODEL}")

prompt = st.chat_input("Ask me anything...")
if prompt:
    with st.chat_message("user"):
        st.write(prompt)
    with st.chat_message("assistant"):
        stream = get_llm().stream([HumanMessage(content=prompt)])
        st.write_stream(chunk.content for chunk in stream)
```

How Streamlit thinks

- Rerun model: every interaction re-executes the whole script from top to bottom.
- `@st.cache_resource` creates the ChatOllama client once and reuses it across reruns.
- `st.chat_input` shows the input box at the bottom and returns the text once, after Enter.
- `st.chat_message("user" | "assistant")` draws a chat bubble with the right avatar.
- `st.write_stream` consumes a generator of text and types it live: a perfect match for `llm.stream`.
- Because of the rerun model, this Day 01 page also forgets old messages. Day 02 solves it with `st.session_state`.

Run it with: `uv run streamlit run app.py` (inside the Day01 folder). Streamlit opens http://localhost:8501.

> **Terminal app vs Streamlit**
> The model logic is identical. Only the "view" changes: input() / print() becomes chat_input / chat_message. Good habit: keep the LLM code separate from the UI code so I can reuse it.

19. Troubleshooting Guide

Symptom	Likely cause	Fix
Connection refused / cannot connect	Ollama server not running	Start the Ollama app or run: ollama serve
404 or model not found, model is installed	BASE_URL ends with /v1 but client is ChatOllama	Section 14: strip /v1
model "qwen3:1.7b" not found	Weights not downloaded	ollama pull qwen3:1.7b
First answer takes very long	Cold start: loading weights	Normal; later calls are faster
Very slow on every call	CPU only, other apps using RAM	Close apps; keep small model; shorter answers
Answer contains <think> text	Qwen3 thinking mode	/no_think, reasoning=False, or strip_think()
ModuleNotFoundError: langchain_ollama	Package missing or wrong environment	uv add langchain-ollama; use uv run
os.getenv returns None	.env not found or load_dotenv() not called first	Run from the project folder; call it before getenv
Answers cut off mid-sentence	num_predict or num_ctx too small	Increase them
Model forgets earlier question	Stateless calls	Send history (Day 02)
Port already in use (11434)	Another Ollama instance running	Stop the other one or change OLLAMA_HOST

Debug checklist, in this order

1. Does `ollama run qwen3:1.7b` answer in the terminal? (model + server OK)
2. Does `curl http://localhost:11434/api/tags` list the model? (HTTP OK)
3. Print `base_url` and `model_name` in Python. Are they what I expect? (config OK)
4. Call `llm.invoke("hi")` in a tiny script. Does it work? (client OK)
5. Only then debug my loop or the UI.

20. Interview Q and A

Q1. What is an LLM?

A neural network (usually a transformer) trained to predict the next token. It outputs a probability distribution over the vocabulary; text is produced by sampling repeatedly.

Q2. What is a token?

A sub-word unit produced by the tokenizer (for example BPE). The model works on integer token ids. Cost, speed and context limits are all measured in tokens.

Q3. Explain temperature.

It divides the logits before softmax. Low T gives focused, repeatable output; high T gives more diverse but riskier output. T near 0 is greedy decoding.

Q4. Temperature vs top-p?

Temperature reshapes the whole distribution; top-p cuts off the unlikely tail and samples from the smallest set whose cumulative probability reaches p. They are often combined.

Q5. Why is attention divided by sqrt(d_k)?

Dot products grow with dimension, which pushes softmax into saturation with tiny gradients. Scaling keeps variance near 1.

Q6. Why run an LLM locally with Ollama?

Privacy (data stays on my machine), zero per-token cost, offline use, full control over versions. Trade-offs: smaller models and limited hardware.

Q7. ChatOllama vs ChatOpenAI for Ollama?

ChatOllama uses the native API and exposes Ollama options (num_ctx, reasoning). ChatOpenAI uses the /v1 OpenAI-compatible API, so code ports to OpenAI easily.

Q8. Why is the chat loop stateless?

The model does not store conversation. Each request must contain all needed

context. Memory is implemented by the application by resending messages.

Q9. What is quantization?

Storing weights in fewer bits (for example 4-bit) to cut memory and speed up inference, with a small accuracy loss.

Q10. What is the KV cache?

Stored keys and values of past tokens so each new token needs only incremental attention work. It grows linearly with context length.

Q11. Chatbot vs agent?

A chatbot answers. An agent loops: it decides, calls tools, observes results, updates memory and continues until the goal is met.

Q12. Why keep config in .env?

Separates configuration and secrets from code, keeps keys out of Git, and lets the same code run in different environments.

Interview tip

Answer in three layers: one-line definition, intuition or example, then the trade-off. Mention that I built it myself with Ollama and LangChain.

21. Formula Sheet and Glossary

Formula sheet

$$\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$$

logits to probabilities

$$\text{softmax}(z / T)$$

temperature scaling

$$P(x_{1..n}) = \prod_t P(x_t \mid x_{<t})$$

autoregressive chain rule

$$\text{Attention} = \text{softmax}(Q K^T / \sqrt{(d_k)}) V$$

scaled dot-product attention

$$\cos(a, b) = (a \cdot b) / (|a| \ |b|)$$

embedding similarity

$$L = - \log P(y \mid \text{context})$$

cross-entropy for one token

$$\text{memory} = \text{parameters} \times \text{bytes per parameter}$$

1.7B at 2 bytes = 3.4 GB; at 0.5 byte = 0.85 GB

$$\text{tokens/s} = \text{eval_count} / (\text{eval_duration} / 10^9)$$

Ollama durations are in nanoseconds

Glossary

Term	Meaning
LLM	Large language model: predicts the next token
Token	Sub-word unit; the model's basic symbol
Embedding	Vector representing a token's meaning
Logit	Raw score before softmax
Context window	Max tokens the model can read at once
Inference	Running a trained model (no learning)
Quantization	Lower-precision weights to save memory
Ollama	Local server that runs open-weight models
Runnable	LangChain interface: invoke, stream, batch
AIMessage	LangChain object holding the model reply
Stateless	No memory between calls
.env	File of environment variables for configuration

uv Fast Python package and project manager

22. Exercises, Checklist, Day 02 Preview

Practice exercises

1. Print response.usage_metadata after each answer and note how input_tokens changes with prompt length.
2. Compute tokens per second for three different questions. Which is fastest and why?
3. Run the same prompt at temperature 0, 0.7 and 1.5, five times each. Compare the answers.
4. Add a SystemMessage that makes the model answer like a Capgemini solution architect. Test it.
5. Add /no_think to a prompt and compare answer time with and without it.
6. Break the config on purpose (keep /v1, stop Ollama, misspell the model). Read each error and match it to Section 19.
7. Write strip_think() unit tests with pytest.
8. Re-implement the loop with ChatOpenAI (Fix C) and compare the code.
9. Extend the TensorFlow or PyTorch attention snippet with a causal mask on a 4×4 score matrix.

Day 01 completion checklist

[ ] Ollama installed and qwen3:1.7b pulled
[ ] uv project created, dependencies added
[ ] .env created and ignored by Git
[ ] Terminal assistant answers questions
[ ] /v1 issue understood and fixed
[ ] Streaming version working
[ ] Streamlit version running at localhost:8501
[ ] Can explain next-token prediction, softmax, temperature and attention aloud

Day 02 preview: conversation memory

Goal: let the agent remember earlier turns. Idea: keep a growing list `history = [SystemMessage, Human, AI, Human, AI, ...]`, send the whole list at every call and append the new AIMessage afterwards. Watch out: history grows, so it will eventually exceed `num_ctx`; later days handle that with trimming and summarising.

Day 02 sketch: memory = resend the list

```
history = [SystemMessage(content="You are a helpful assistant.")]

while True:
    q = input("You: ")
    if q.lower() == "exit":
        break
    history.append(HumanMessage(content=q))
    reply = llm.invoke(history)          # send EVERYTHING
    history.append(reply)                # AIMessage goes back in
    print("AI:", reply.content)
```

Final takeaway

An LLM is a next-token probability machine. Ollama serves it, LangChain wraps it, my loop talks to it. Everything else in AI agent engineering is built by adding memory, tools and control flow around this one call.

- Chandan Chaudhari | github.com/chandanc5525 -